Coverage-Based Testing: `coverage.py`, `pytest-cov`, and Coverage Analysis

Week 7 Lecture — SE 413/513: Software Testing

Dr. Kamrul Hasan

Grand Valley State University

Fall 2026

Today's Plan (60 minutes)

Segment	Time
1. Introduction & Objectives	3 min
2. What Coverage-Based Testing Is (and Isn't)	5 min
3. Statement, Branch, and Function Coverage	6 min
4. Measuring with `coverage.py` (live demo)	10 min
5. `pytest-cov`, Configuration & Enforcing a Minimum	8 min
6. Interpreting Reports: Columns, Arrows, HTML	10 min
7. From Report to Tests: Closing the Gaps	7 min
8. What Coverage Cannot Tell You	8 min
9. Project Connection & Wrap-Up	3 min
Total	**60 min**

Based on this week's reading, *Coverage-Based Testing*. Every report on these slides is real output from `coverage.py` 7.16 / `pytest-cov` on the reading's `checkout.py` example.

Learning Objectives

By the end of today's class, you should be able to:

- Explain what coverage-based testing is, and why coverage measures **adequacy**, not correctness.
- Define and compute **statement**, **branch**, and **function** coverage.
- Run `coverage.py` and `pytest-cov`, and produce terminal and HTML reports.
- Read every column of a report — including `BrPart` and arrows like `16->18`.
- Turn coverage gaps into meaningful, behavior-level tests.
- Explain the limitations of coverage, and use it responsibly on your project.

Part 1

What Coverage-Based Testing Is (and Isn't)

~5 minutes

From Criteria to Tools

From earlier weeks: Week 5, Week 6

Weeks 5–6: the *criteria*

- CFGs; statement, branch, path coverage
- Path $\Rightarrow$ branch $\Rightarrow$ statement
- Data flow: All-Defs, All-Uses, All-DU-Paths

Week 7: the *practice*

- Measuring coverage with real tools
- Reading reports *accurately*
- Deciding what to do about the gaps

Key Idea

Coverage-based testing uses how much of the code's structure a test suite exercises to *find and close gaps*. It answers one narrow, very practical question: *"Which parts of the code did my tests never even run?"*

Coverage Is an *Adequacy* Criterion

Three jobs coverage does well:

1. **Finding gaps** — often error handling, rare options, new code.
2. **Guiding new tests** — each gap points to an untried input.
3. **Guarding the suite over time** — tracked in CI, it flags code merged without tests.

Low coverage
reliable bad news

High coverage
at best, weak good news

Coverage measures **execution**, not **verification**: a line is "covered" the moment it runs, whether or not anything checks it.

Part 2

Statement, Branch, and Function Coverage

~6 minutes

Three Criteria Python Tools Report

$$\textbf{Statement} = \frac{\text{statements executed}}{\text{executable statements}} \qquad \textbf{Branch} = \frac{\text{outcomes taken}}{\text{branch outcomes}} \qquad \textbf{Function} = \frac{\text{functions called}}{\text{functions}}$$

Criterion	**Catches**	**In `coverage.py`**
Function	Whole functions with no test	Per-function table in HTML report (v7.5+)
Statement	Lines never executed	Default
Branch	Decision outcomes never taken (`if` with no `else`!)	`--branch` — **off by default**

Tip

Each `if`, `elif`, `while`, and `for` has **two** outcomes. For a `for` loop: "enter the body" and "exit the loop."

Criteria Python Tools Do *Not* Measure

Criterion	Requires that tests. . .	Seen in
Condition	make every boolean sub-condition both true and false	—
MC/DC	show each condition *independently* changes the decision	Avionics standards
Path	execute every entry-to-exit path	Week 5 (often infeasible)
Data-flow	exercise def-use pairs	Week 6

Keep **condition coverage** in mind — we'll meet its absence again in Part 7.

Part 3

Measuring with `coverage.py`

~10 minutes, including live demo

The Running Example: `checkout.py`

```
4  def subtotal(items):
5      total = 0.0
6      for price, qty in items:
7          if qty < 0:
8              raise ValueError("quantity cannot be
       negative")
9          total += price * qty
10     return total
11
12
13 def apply_coupon(amount, code):
14     if code == "SAVE10":
15         amount = amount * 0.90
16     elif code == "FLAT5":
17         amount = max(amount - 5, 0)
18     return round(amount, 2)
```

```
21 def shipping_fee(amount, is_member):
22     if is_member or amount >= 50:
23         return 0.0
24     return 7.99
25
26
27 def format_receipt(amount):
28     return f"Total due: ${amount:.2f}"
```

Line 1 is a module docstring; numbering matches the reading and the reports.

A Typical First Test Suite (v1)

```python
def test_subtotal_two_items():
    assert subtotal([(10.0, 2), (5.0, 1)]) == 25.0

def test_save10_coupon():
    assert apply_coupon(100.0, "SAVE10") == 90.0

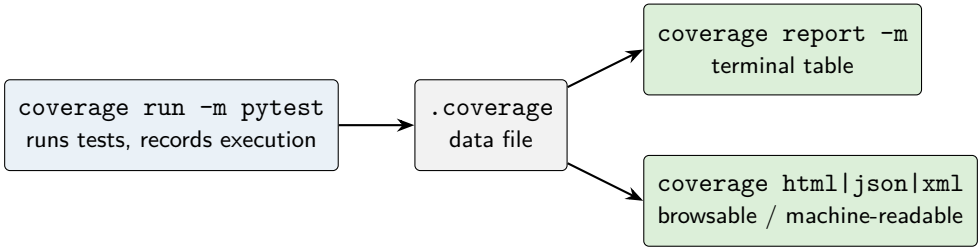
def test_member_ships_free():
    assert shipping_fee(20.0, True) == 0.0
```

- One happy-path test per function...
- ...and `format_receipt()` forgotten.
- All 3 tests **pass**.

Quick Check — Think, Then Discuss

Before we measure: guess the statement coverage of `checkout.py`.

The coverage.py Workflow



Key Idea

Measure once, report many ways. Measuring (`run`) and reporting (`report`, `html`, ...) are separate steps on the same saved data.

Step 1: The Basic Report

```
$ pip install coverage pytest
$ coverage run -m pytest tests/test_checkout_v1.py
$ coverage report
Name                          Stmts   Miss  Cover
-----------------------------------------------
checkout_app/__init__.py          0      0   100%
checkout_app/checkout.py         19      5    74%
tests/__init__.py                 0      0   100%
tests/test_checkout_v1.py         7      0   100%
-----------------------------------------------
TOTAL                            26      5    81%
```

Common Pitfall

81% is inflated: test files are always ∼100% executed. Measure only your app with `--source=checkout_app`. The honest number for `checkout.py`: **74%** (14/19 statements).

Step 2: What Was Missed (`-m`)

```
$ coverage report -m
Name                          Stmts   Miss  Cover   Missing
-----------------------------------------------------------
checkout_app/checkout.py         19      5    74%   8, 16-17, 24, 28
```

Lines	Untested behavior
8	negative quantity raises `ValueError`
16–17	the `FLAT5` coupon
24	paid shipping (`return 7.99`)
28	all of `format_receipt()`

A percentage is a score; **the `Missing` column is a to-do list.**

Step 3: Turn On Branch Coverage

```
$ coverage run --branch --source=checkout_app -m pytest tests/test_checkout_v1.py
$ coverage report -m
Name                          Stmts   Miss Branch BrPart  Cover   Missing
-------------------------------------------------------------------------
checkout_app/checkout.py         19      5     10      3    66%   8, 16-17, 24, 28
```

Quick Check — Think, Then Discuss

The tests didn't change, but coverage **dropped** from 74% to 66%. Why?

Step 3: Turn On Branch Coverage

```
$ coverage run --branch --source=checkout_app -m pytest tests/test_checkout_v1.py
$ coverage report -m
Name                          Stmts   Miss Branch BrPart  Cover   Missing
-------------------------------------------------------------------------
checkout_app/checkout.py         19      5     10      3    66%   8, 16-17, 24, 28
```

Quick Check — Think, Then Discuss

The tests didn't change, but coverage **dropped** from 74% to 66%. Why?

Common Pitfall

The report now counts something it ignored before: **branch outcomes**. By default `coverage.py` reports **statement coverage only** — the more flattering number. Always say which one you're quoting.

Live Demo Checklist

1. `coverage run -m pytest` → `coverage report`
2. Add `--source` — watch the test files disappear from the table.
3. Add `-m` — map each missing line to the source.
4. Add `--branch` — watch 74% become 66%.
5. `coverage html` — open `htmlcov/index.html` (we'll read it in Part 5).

Tip

Follow along: the code and both test suites are in the reading (Sections 5 and 10). Run them yourself as Reading Exercise 3.

Part 4

`pytest-cov`, Configuration & Enforcing a Minimum

~8 minutes

pytest-cov: Coverage as pytest Options

```
$ pip install pytest-cov
$ pytest --cov=checkout_app --cov-branch --cov-report=term-missing
============================== tests coverage ==============================
Name                         Stmts   Miss Branch BrPart  Cover   Missing
-------------------------------------------------------------------------
checkout_app/__init__.py         0      0      0      0   100%
checkout_app/checkout.py        19      5     10      3    66%   8, 16-17, 24, 28
-------------------------------------------------------------------------
TOTAL                           19      5     10      3    66%
3 passed in 0.03s
```

Same engine $\Rightarrow$ **same numbers**. Advantages: one command, CI-friendly, handles parallel runs (`pytest-xdist`).

Key `pytest-cov` Options

Option	Effect
`--cov=PKG`	Measure package `PKG` (bare `--cov`: use config file)
`--cov-branch`	Enable branch measurement
`--cov-report=term-missing`	Terminal table with `Missing` column
`--cov-report=html`	Write `htmlcov/`
`--cov-report=xml` / `json`	Machine-readable (CI services, scripts)
`--cov-fail-under=N`	Fail the run if total coverage $<$ `N`%
`--cov-context=test`	Record *which test* ran each line

Several `--cov-report` options can be combined in one run.

Make It Permanent: pyproject.toml

```toml
[tool.coverage.run]
branch = true
source = ["checkout_app"]

[tool.coverage.report]
show_missing = true
fail_under = 90
exclude_also = [
    "if __name__ == .__main__.:",
]
```

- Read by **both** coverage and pytest-cov.
- You, your teammates, and CI all get the **same** numbers.
- Branch coverage on by default — no more flattering numbers.

```
$ coverage report
...
Coverage failure: total of 66
  is less than fail-under=90
```

Enforcing a Minimum in CI

```
$ pytest --cov=checkout_app --cov-branch --cov-fail-under=80
FAIL Required test coverage of 80% not reached. Total coverage: 65.52%
3 passed in 0.03s
$ echo $?
1
```

All 3 **tests passed** — but the **command failed** (exit status 1). In CI, that fails the build:

```
- name: Run tests with coverage          # GitHub Actions step
  run: |
    pip install pytest pytest-cov
    pytest --cov --cov-branch --cov-report=xml --cov-fail-under=80
```

Tip

A threshold is a **floor** against regressions, not a target. More on this in Part 7 — and coverage in CI pipelines in the Continuous Testing week.

Part 5

Interpreting Reports: Columns, Arrows, HTML

~10 minutes

Every Column, Explained

Column	Meaning
`Stmts`	executable statements
`Miss`	never executed
`Branch`	branch outcomes (2/decision)
`BrPart`	decisions that took only **one** outcome
`Cover`	combined % (below)
`Missing`	lines + arcs never taken

How 66% is computed (v1 suite):

$$\frac{\text{stmts run} + \text{outcomes taken}}{\text{stmts} + \text{outcomes}} = \frac{14+5}{19+10} \approx 65.5\%$$

- Statement alone: $14/19 \approx 74\%$
- Branch alone: $5/10 = 50\%$

10 outcomes = 5 decisions × 2: the `for` on line 6, and the conditions on lines 7, 14, 16, 22.

The 3 Partial Branches

Line	Decision	Always...	Because
7	`if qty < 0`	false	no negative quantity tested
14	`if code == "SAVE10"`	true	only SAVE10 tested
22	`if is_member or amount >= 50`	true	only a member tested
16	`elif code == "FLAT5"`	*never ran*	**missing**, not partial

Key Idea

Partial = the decision ran but went only one way. **Missing** = it never ran at all. Both of line 16's outcomes count as missed branches.

Reading the `Missing` Column: Arrows

8	line 8 never ran
16–17	lines 16 through 17 never ran
16->18	line 16 ran, but never jumped *directly* to line 18

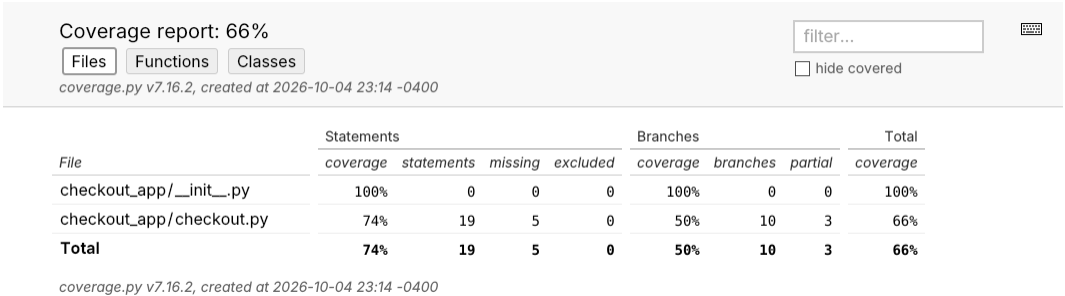
Why no arrows in v1? Each missed arc ended at a line already listed (7→8 is implied by "8"). Test only `SAVE10` and `FLAT5`:

```
checkout_app/checkout.py      19      10      10       1    41%   5-10, 16->18, 22-24, 28
```

Key Idea

16->18 = "neither coupon matched." Every a->b becomes a question: *what input decides line a the way that leads straight to b?* Here: **an unrecognized coupon code**.

The HTML Report: Index



Coverage report: 66%

Files | Functions | Classes

filter...

hide covered

coverage.py v7.16.2, created at 2026-10-04 23:14 -0400

	Statements				Branches			Total
File	*coverage*	*statements*	*missing*	*excluded*	*coverage*	*branches*	*partial*	*coverage*
checkout_app/__init__.py	100%	0	0	0	100%	0	0	100%
checkout_app/checkout.py	74%	19	5	0	50%	10	3	66%
Total	**74%**	**19**	**5**	**0**	**50%**	**10**	**3**	**66%**

coverage.py v7.16.2, created at 2026-10-04 23:14 -0400

- Statement (74%), branch (50%), and total (66%) **reported separately**.
- `coverage html` or `pytest --cov-report=html` → open `htmlcov/index.html`.

The HTML Report: Annotated Source


Coverage for **checkout_app/checkout.py**: 66%

19 statements | 14 run | 5 missing | 0 excluded | 3 partial

« prev ^ index » next coverage.py v7.16.2, created at 2026-10-04 23:14 -0400

```
1  """A small checkout module used in the Week 7 coverage examples."""
2
3
4  def subtotal(items):
5      total = 0.0
6      for price, qty in items:
7          if qty < 0:                                   7 ↛ 8
8              raise ValueError("quantity cannot be negative")
9          total += price * qty
10     return total
11
12
13 def apply_coupon(amount, code):
14     if code == "SAVE10":                              14 ↛ 16
15         amount = amount * 0.90
16     elif code == "FLAT5":
17         amount = max(amount - 5, 0)
18     return round(amount, 2)
19
20
21 def shipping_fee(amount, is_member):
22     if is_member or amount >= 50:                     22 ↛ 24
23         return 0.0
24     return 7.99
25
26
27 def format_receipt(amount):
28     return f"Total due: ${amount:.2f}"
```

« prev ^ index » next coverage.py v7.16.2, created at 2026-10-04 23:14 -0400

Green bar: executed

Red: never executed

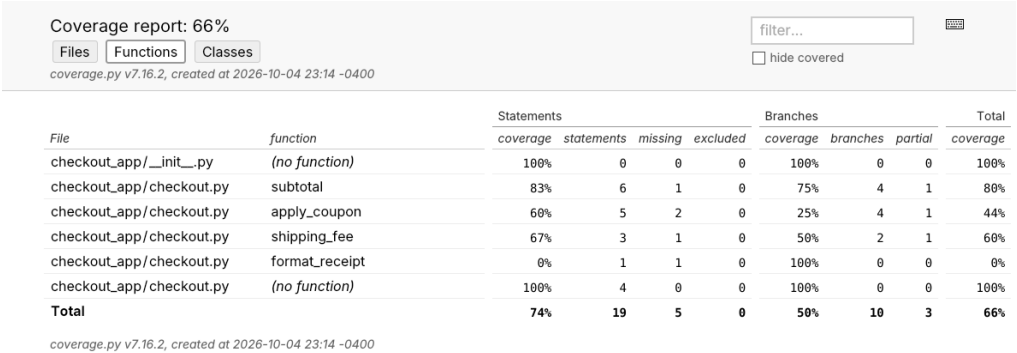
Yellow: partial branch; missed arc at right ($7 \nrightarrow 8$)

Hover a yellow line: *"line 7 didn't jump to line 8 because the condition on line 7 was never true."*

Tip

Toggle off *run*: what remains is your to-do list.

The HTML Report: Function Coverage



Coverage report: 66%

Files | Functions | Classes

coverage.py v7.16.2, created at 2026-10-04 23:14 -0400

filter...

hide covered

		Statements				Branches			Total
File	function	coverage	statements	missing	excluded	coverage	branches	partial	coverage
checkout_app/__init__.py	(no function)	100%	0	0	0	100%	0	0	100%
checkout_app/checkout.py	subtotal	83%	6	1	0	75%	4	1	80%
checkout_app/checkout.py	apply_coupon	60%	5	2	0	25%	4	1	44%
checkout_app/checkout.py	shipping_fee	67%	3	1	0	50%	2	1	60%
checkout_app/checkout.py	format_receipt	0%	1	1	0	100%	0	0	0%
checkout_app/checkout.py	(no function)	100%	4	0	0	100%	0	0	100%
Total		**74%**	**19**	**5**	**0**	**50%**	**10**	**3**	**66%**

coverage.py v7.16.2, created at 2026-10-04 23:14 -0400

Function coverage $= 3/4 = 75\%$: `format_receipt` is at 0%. `apply_coupon` (44%) is the weakest function *with* tests. "(no function)" rows = module-level code (imports, `def` lines).

Part 6

From Report to Tests: Closing the Gaps

~7 minutes

Each Gap Is a Question About Behavior

Gap	Behavior never checked	New test
8 / 7 $\nrightarrow$ 8	negative qty rejected	`subtotal([(10.0, -1)])` raises `ValueError`
16–17	`FLAT5` subtracts $5	`apply_coupon(100.0, "FLAT5") == 95.0`
(17, `max`)	`FLAT5` never negative	`apply_coupon(3.0, "FLAT5") == 0.0`
16 $\rightarrow$ 18	unknown code: no change	`apply_coupon(100.0, "BOGUS") == 100.0`
24 / 22 $\nrightarrow$ 24	non-member pays	`shipping_fee(20.0, False) == 7.99`
28	receipt format	`format_receipt(12.5) == "Total due: $12.50"`

Common Pitfall

Row 3: the `max` floor is *one line* — any `FLAT5` test covers it. Coverage will never ask for the $3 test; **the specification must**.

The v2 Suite: 100% Across the Board

```python
@pytest.mark.parametrize("amount, code, expected", [
    (100.0, "SAVE10", 90.0),
    (100.0, "FLAT5", 95.0),
    (3.0, "FLAT5", 0.0),
    (100.0, "BOGUS", 100.0),
])
def test_apply_coupon(amount, code, expected):
    assert apply_coupon(amount, code) == expected

def test_non_member_small_order_pays_shipping():
    assert shipping_fee(20.0, False) == 7.99
# ... plus negative-qty, member, receipt tests
```

```
Name          Stmts Miss Branch BrPart  Cover
-----------------------------------------------
checkout.py      19    0     10      0   100%
9 passed in 0.04s
```

Quick Check — Think, Then Discuss

100% statement, branch, *and* function coverage. Is `checkout.py` fully tested?

Part 7

What Coverage Cannot Tell You

~8 minutes

Limitation 1: Execution Is Not Verification

```python
def test_runs_everything():
    for code in ["SAVE10", "FLAT5", "NONE"]:
        apply_coupon(100.0, code)
    shipping_fee(20.0, True)
    shipping_fee(20.0, False)
    format_receipt(1)
    subtotal([(1.0, 1)])
    try:
        subtotal([(1.0, -1)])
    except ValueError:
        pass
```

- **100% statement and branch coverage** — verified.
- **Zero assertions.**
- Passes if `apply_coupon` returns `None`, if `FLAT5` *adds* \$5, if shipping is always free...

Coverage tools see *execution*; they can't see assertions.

Limitations 2 & 3: Conditions and Boundaries

2. Branches are not conditions. `if is_member or amount >= 50`

- v2 takes the decision both ways — but `or` **short-circuits**: `amount >= 50` has **never been true**.
- "Free shipping over $50" is unverified, and the report shows **no gap**. (Condition coverage / MC/DC would.)

3. Coverage can't see boundaries.

From earlier weeks: Week 3, Week 5

- We changed `>= 50` to `> 50` and re-ran v2: **9 passed, 100% coverage**.
- Only `shipping_fee(50.0, False) == 0.0` — **boundary-value analysis** — catches it.

Tip

Planting a small bug and checking whether a test fails is **mutation testing** (e.g., `mutmut`) — it measures *fault detection*, not execution. An advanced topic later in the course.

Limitation 4 and What the Research Says

4. Coverage says nothing about missing code. If the spec says coupons are case-insensitive and nobody implemented it, there's *no line to be uncovered*. Only black-box testing finds missing features.

- **Inozemtseva & Holmes (ICSE 2014):** controlling for suite size, coverage is only weakly-to-moderately correlated with fault detection.
- Good indicator of what's *untested*; poor target to optimize.

Real-World Case Study

Coverage at Google (Ivanković et al., FSE 2019): measured across 1B+ lines and shown in code review. Developers valued **coverage of changed lines** most. Published guidance offers rough bands — ∼60% acceptable, 75% commendable, 90% exemplary — and warns against treating any number as the goal.

Best Practices (Short Version)

1. **Measure branches**, and only your application code.
2. **Read the gaps, not the percentage.**
3. **Close gaps with behavior-level tests and real assertions.**
4. **Combine with black-box design:** EP/BVA chooses *values*; coverage confirms nothing was left untouched.
5. **Set a floor, not a target**; watch coverage of *changed* code.
6. **Exclude sparingly and visibly** (`# pragma: no cover`) — every exclusion should survive code review.

Common Pitfall

Goodhart's Law: once the number becomes the target, it stops measuring what you cared about.

Part 8

Project Connection & Wrap-Up

~3 minutes

Using Coverage in Your Project's Preliminary Results

- State the **exact command and configuration** (statement or branch? which packages?).
- Report coverage **per module**, not only the total.
- For key gaps: **test it, exclude it (with a reason), or remove it** as dead code.
- Pair the numbers with tests designed **from the specification** — evidence your suite does more than execute lines.
- **Track A:** measure the existing suite *first* — a baseline makes your contribution measurable.

Key Takeaways

1. **Coverage finds what you did not test**; it cannot certify what you did.
2. **Report branch coverage** — statement-only is the flattering number.
3. `coverage.py` **measures, then reports**; `pytest-cov` does both inside `pytest`.
4. **The `Missing` column and HTML report are the value**: every red line and every `a->b` is an untested behavior.
5. **100% is not "done"**: assertions, sub-conditions, boundaries, and missing features are invisible to coverage.

What We Didn't Cover Today

In the interest of pacing, these are in the reading but were **not** covered (or only briefly) in lecture — you are responsible for them as reading:

- Section 6.5, the full table of `coverage` commands (`erase`, `combine`, `--skip-covered`) — also in backup slides.
- Section 8.4, machine-readable reports (JSON, Cobertura XML).
- Section 12, the full Best Practices list; Section 13, Common Mistakes (e.g., stale data).
- Sections 15–16, Mental Model and Summary tables (backup slides).
- Sections 17–18, all Practice Exercises (**homework**) and Discussion Questions.

The boundary-value and shipping-weight connections come from Week 3 and Week 5.

Questions?

Homework: Reading Exercises 1–5

Run the `checkout.py` example yourself (Exercise 3)

Dr. Kamrul Hasan | Grand Valley State University

Backup Slides

Reference material only — not part of the 60-minute lecture

Backup: Other `coverage` Commands (Reading, Section 6.5)

Command	Purpose
`coverage run -m pytest`	Run tests, record data in `.coverage`
`coverage report -m`	Terminal summary with missing lines
`coverage report --skip-covered`	Hide files already at 100%
`coverage html`	Browsable report in `htmlcov/`
`coverage json` / `xml`	Machine-readable reports (XML = Cobertura format)
`coverage erase`	Delete previously collected data
`coverage combine`	Merge data from several runs (e.g., parallel processes)

Backup: Full Best Practices (Reading, Section 12)

1. **Measure branches, not just statements.** Turn branch measurement on permanently.
2. **Measure the right code.** Use `source` / `--cov=PKG`.
3. **Read the gaps, not the percentage.**
4. **Turn each gap into a behavior-level test with real assertions.**
5. **Combine coverage with black-box design.**
6. **Set a floor, not a target**; raise it gradually; watch changed code.
7. **Exclude sparingly and visibly**: e.g., `if __name__ == "__main__":` launchers, platform-specific defensive code. Excluding code because it's *hard* to test hides exactly the gaps coverage exists to show.
8. **Investigate 0% functions and long red blocks first.**

Common mistakes: statement-only numbers quoted as "coverage"; test files in the total; tests written to hit lines; treating 100% as done; stale data (`coverage erase`).

Backup: A Practical Mental Model (Reading, Section 15)

A coverage percentage? *"Which criterion, over which code — and where are the gaps?"*

A red line in the HTML report? *"What behavior does this line implement, and what input would make it run?"*

`a->b` in the `Missing` column? *"The decision on line `a` was never decided in the direction that leads to `b`. What input decides it that way?"*

100%? *"Every line ran. Did my assertions check the results, did I test the boundaries, and did every sub-condition matter?"*

Backup: Summary (Reading, Section 16)

Criterion	Catches	Misses
Function	Whole functions with no test	Almost everything inside a function
Statement	Lines never executed	`false` outcomes of an `if` with no `else`
Branch	Decision outcomes never taken	Short-circuited sub-conditions, boundary values, weak assertions, missing code

Key Idea

Coverage finds what you did not test; it cannot certify what you did. Use it as a flashlight for gaps, then fill them with specification-based tests backed by real assertions.

Backup: Practice Exercises (Homework)

- **Ex. 1:** 40 statements, 12 branch outcomes; suite executes 34 statements and 8 outcomes. Compute statement, branch, and combined Cover.
- **Ex. 2:** A suite calling `subtotal([])`, `apply_coupon(100.0, "FLAT5")`, `shipping_fee(80.0, False)`, `format_receipt(1)` gives:

```
checkout_app/checkout.py      19      5     10      4    62%   7-9, 15, 16->18, 24
```

  Explain each entry; identify the 4 partial branches; verify 62%; write a test for each entry.
- **Ex. 3:** Reproduce 74% / 66%, find the yellow lines in HTML, reach 100% branch one test at a time.
- **Ex. 4:** From v2, make `amount >= 50` true at least once, and detect the `>=`→`>` change.
- **Ex. 5:** Argue for/against `--cov-fail-under=100` in CI; propose an alternative.

Backup: Discussion Questions (Reading, Section 18)

1. Why is coverage an *adequacy* criterion rather than a *correctness* criterion?
2. Why did `--branch` *lower* the reported percentage for the same suite?
3. What is a partial branch, and why is line 16 missing rather than partial under v1?
4. Why does the `Missing` column sometimes show arrows and sometimes not?
5. Function coverage is easy to achieve. Why is it still worth looking at?
6. How can a suite reach 100% branch coverage while a sub-condition is never true? Which criterion catches this?
7. Why can coverage never reveal a missing feature?
8. Risks of a high project-wide threshold? What does focusing on *changed* code offer?
9. When is `# pragma: no cover` justified, and when is it a warning sign?
10. How do coverage-based and specification-based testing complement each other?

Backup: References (from the Reading)

- Batchelder, N., et al. (2026). *Coverage.py Documentation*. `https://coverage.readthedocs.io/`
- pytest-cov contributors (2026). *pytest-cov Documentation*. `https://pytest-cov.readthedocs.io/`
- Zhu, H., Hall, P. A. V., & May, J. H. R. (1997). Software unit test coverage and adequacy. *ACM Computing Surveys*, 29(4), 366–427.
- Inozemtseva, L., & Holmes, R. (2014). Coverage is not strongly correlated with test suite effectiveness. *ICSE 2014*, 435–445.
- Ivanković, M., Petrović, G., Just, R., & Fraser, G. (2019). Code coverage at Google. *ESEC/FSE 2019*, 955–963.
- Arguelles, C., Ivanković, M., & Bender, A. (2020). *Code Coverage Best Practices*. Google Testing Blog.
- Marick, B. (1999). How to misuse code coverage.
- Ammann, P., & Offutt, J. (2016). *Introduction to Software Testing* (2nd ed.); Aniche, M. (2022). *Effective Software Testing*; ISTQB CTFL v4.0.1.
- Hasan, K. (2026). *Software Testing*. Grand Valley State University. `https://github.com/mdkamrulhasan/software-testing`